

SIR

ORKESTRACIJA MIKROSERVISA U KUBERNETESU

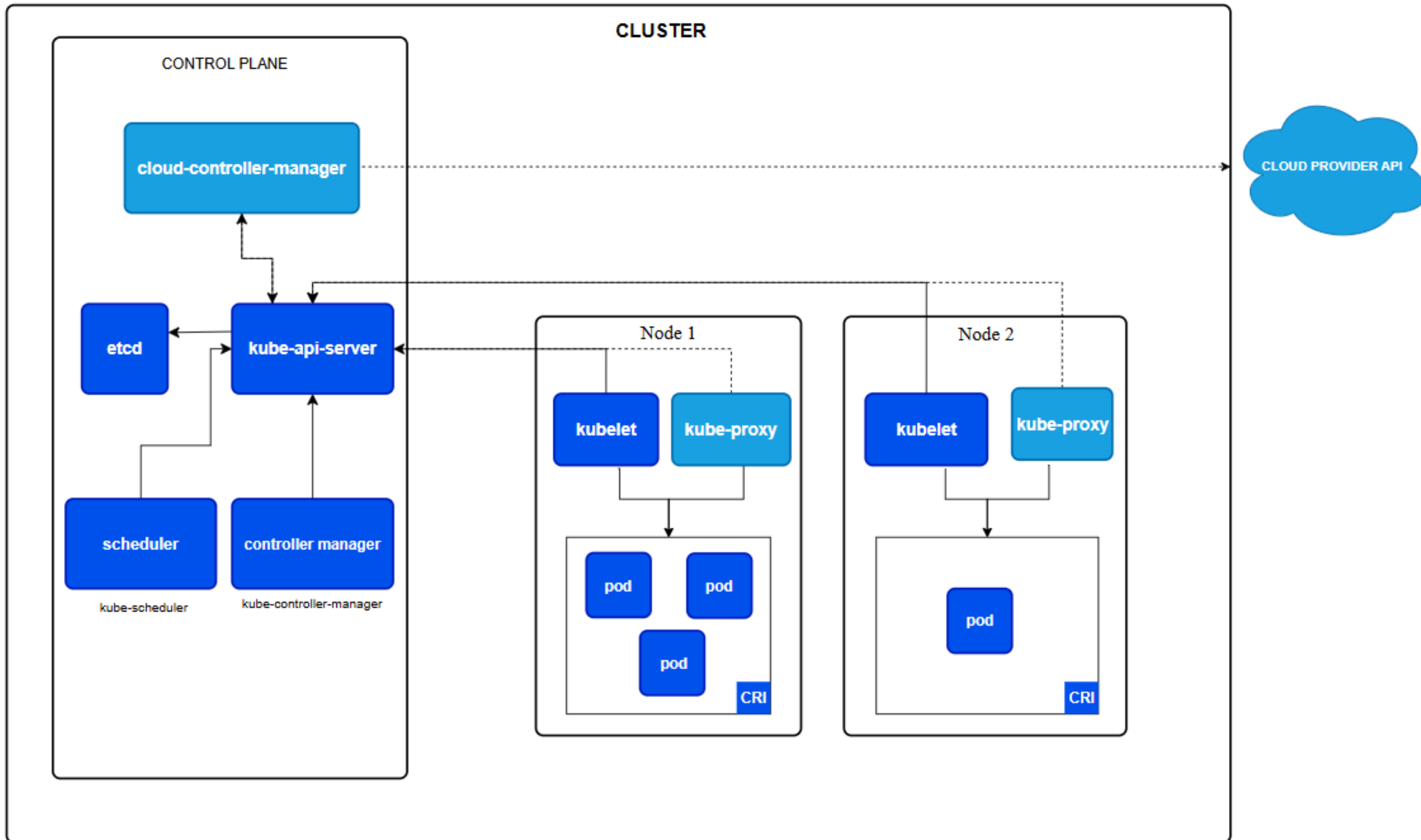
Veljko Veljovic

Kubernetes

- Open source platforma namenjena orkestraciji kontejnera.
- Razvijena sa ciljem automatizacije upravljanja.
- Omogućava automatsko skaliranje aplikacija prema potrebama sistema.
- Pruža visoku dostupnost i pouzdanost kroz automatski oporavak servisa.
- Automatsko raspoređivanje aplikacija na servere (nodove).
- Omogućava efikasno iskorišćavanje računarskih resursa



Kubernetes klaster



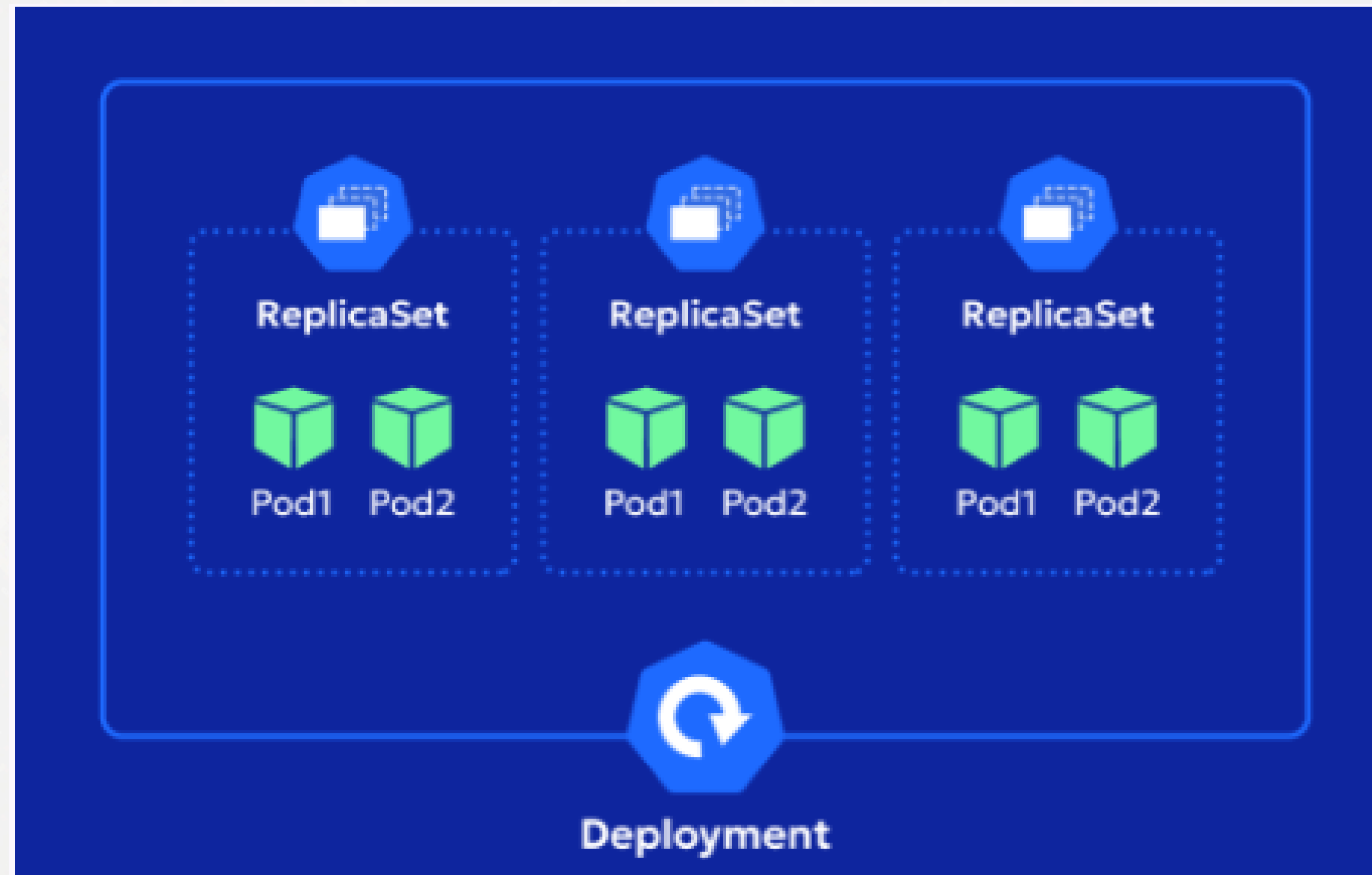
Pod i ReplicaSet

Pod:

- Najmanja izvršna jedinica u Kubernetesu.
- Predstavlja jedan ili više kontejnera koji se izvršavaju zajedno na istom nodu.
- Apstrakcija iznad kontejnera, da K8S nije direktno vezan za određeni CR.

ReplicaSet

- Obezbeđuje da u svakom trenutku postoji određeni broj kopija jednog Pod-a.
- Automatski kreira nove Pod-ove ako neki postojeći prestane sa radom.
- Održava željeno stanje aplikacije definisano brojem replika



Deployment

- Automatski kreira i upravlja ReplicaSet-ovima koji dalje upravljaju Pod-ovima.
- Omogućava definisanje željenog stanja aplikacije (broj replika i verzija aplikacije).
- Omogućava ažuriranje aplikacije bez prekida rada.
- Automatski vraća prethodnu verziju aplikacije ako nova verzija izazove probleme.

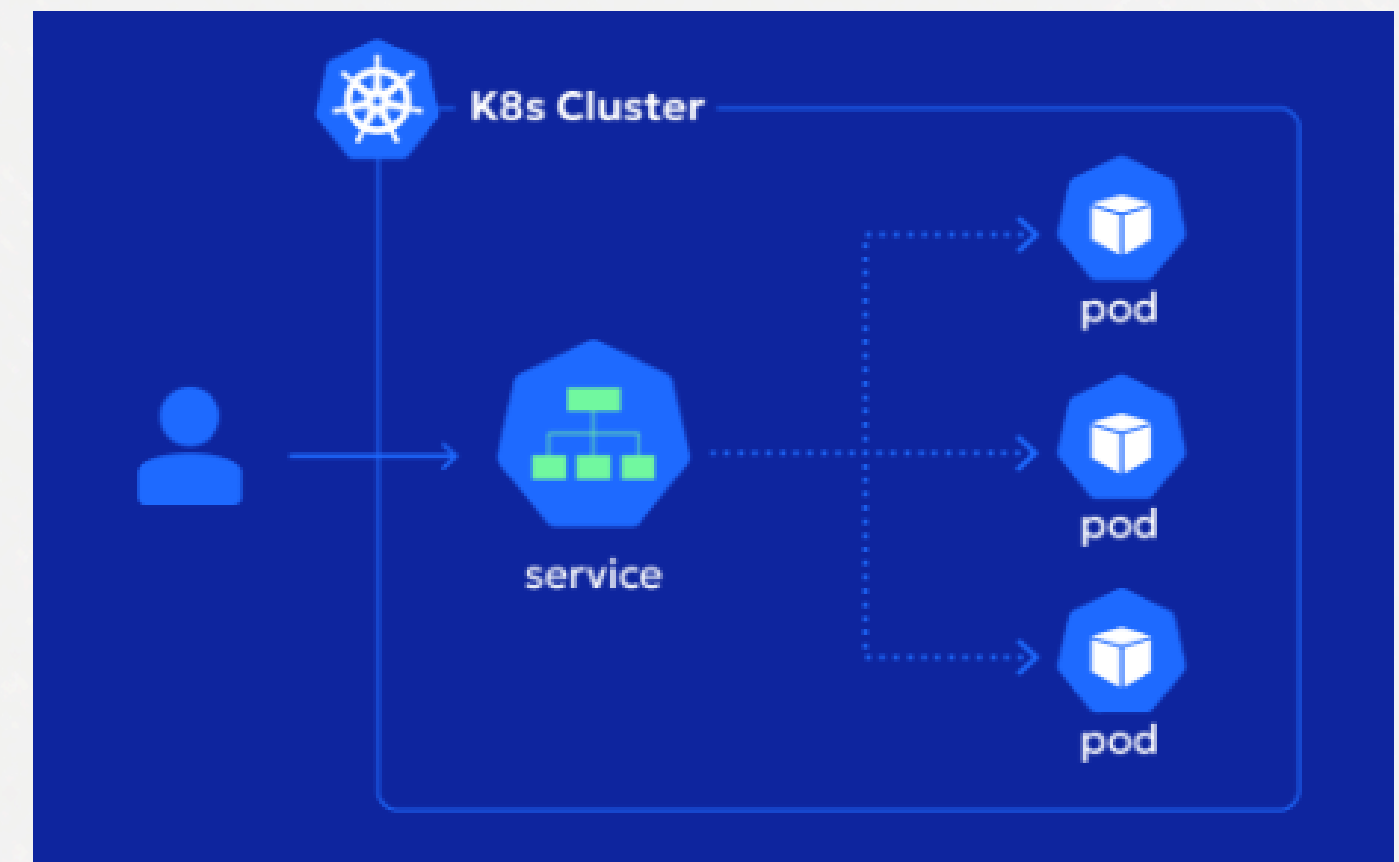
```
consumer > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: consumer-deployment
5    labels:
6      app: consumer-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: consumerapp
12   template:
13     metadata:
14       labels:
15         app: consumerapp
16     spec:
17       containers:
18         - name: consumerapp
19           image: consumer:latest
20           imagePullPolicy: Never
21           ports:
22             - containerPort: 8080
```

Service

- Omogućava stabilnu adresu za Pod.
- Povezuje se sa podovima pomoću label selector-a.
- Prosleđuje zahteve ka odgovarajućim Pod-ovima i omogućava raspodelu saobraćaja između više replika.

```
36 ---
37 apiVersion: v1
38 kind: Service
39 metadata:
40   name: dashboard-service
41   labels:
42     app: dashboardapp
43 spec:
44   type: NodePort
45   selector:
46     app: dashboardapp
47   ports:
48     - protocol: TCP
49       port: 8080
50       targetPort: 8080
```

- ClusterIP - podrazumevani tip servisa i omogućava komunikaciju između podova samo unutar klastera.
- NodePort - izlaže servis na određenom portu Worker noda.
- LoadBalancer - omogućava javni pristup preko cloud LB.

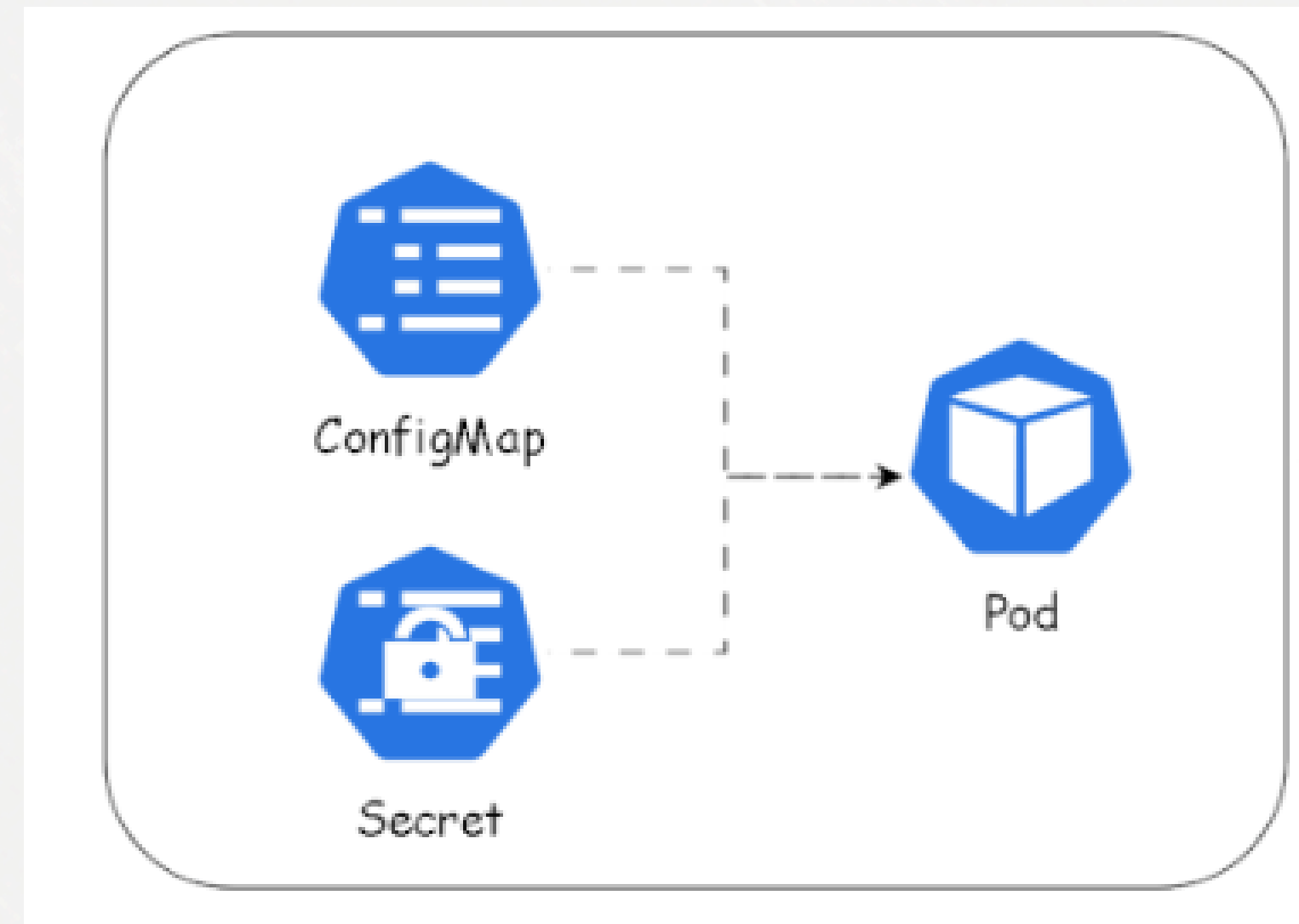


ConfigMap i Secret

ConfigMap

```
consumer > config-map.yaml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: consumer-config-map
5  data:
6    rabbitmq_host: "rabbitmq-service"
7    seq_host: "http://seq-service:5341/"
8
```

- Kubernetes objekt koji služi za čuvanje konfiguracionih podataka aplikacije.
- Omogućava odvajanje konfiguracije od samog koda aplikacije..



```
consumer > secret.yaml
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: consumer-secret
5  type: Opaque
6  data:
7    RabbitMQ__Username: dXNlcg==
8    RabbitMQ__Password: cGFzc3dvcmQ=
9    RabbitMQ__HostString: YW1xcDovL3VzZXI6cGFzc3dvcmRAcmF3
10   ConnectionStrings__Database: SG9zdD1wb3N0Z3JlcztQb3J0P
```

Secret

- Kubernetes objekt namenjen za čuvanje osjetljivih podataka.
- Podaci u Secret-u su Base64 kodirani.

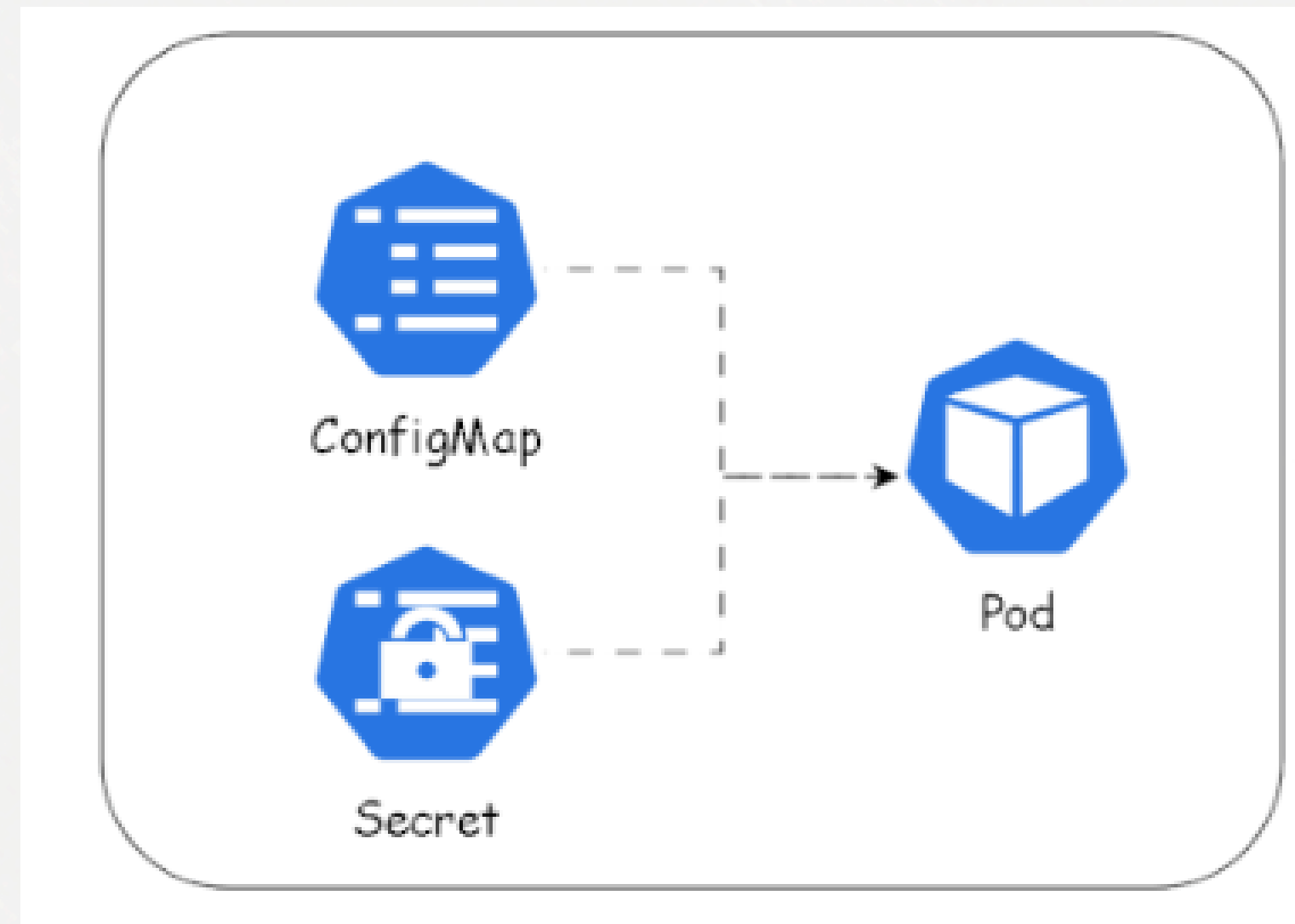
ConfigMap i Secret

ConfigMap

```
consumer > config-map.yaml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: consumer-config-map
5  data:
6    rabbitmq_host: "rabbitmq-service"
7    seq_host: "http://seq-service:5341/"
8
```

- Kubernetes objekat koji služi za čuvanje konfiguracionih podataka aplikacije.
- Omogućava odvajanje konfiguracije od samog koda aplikacije..

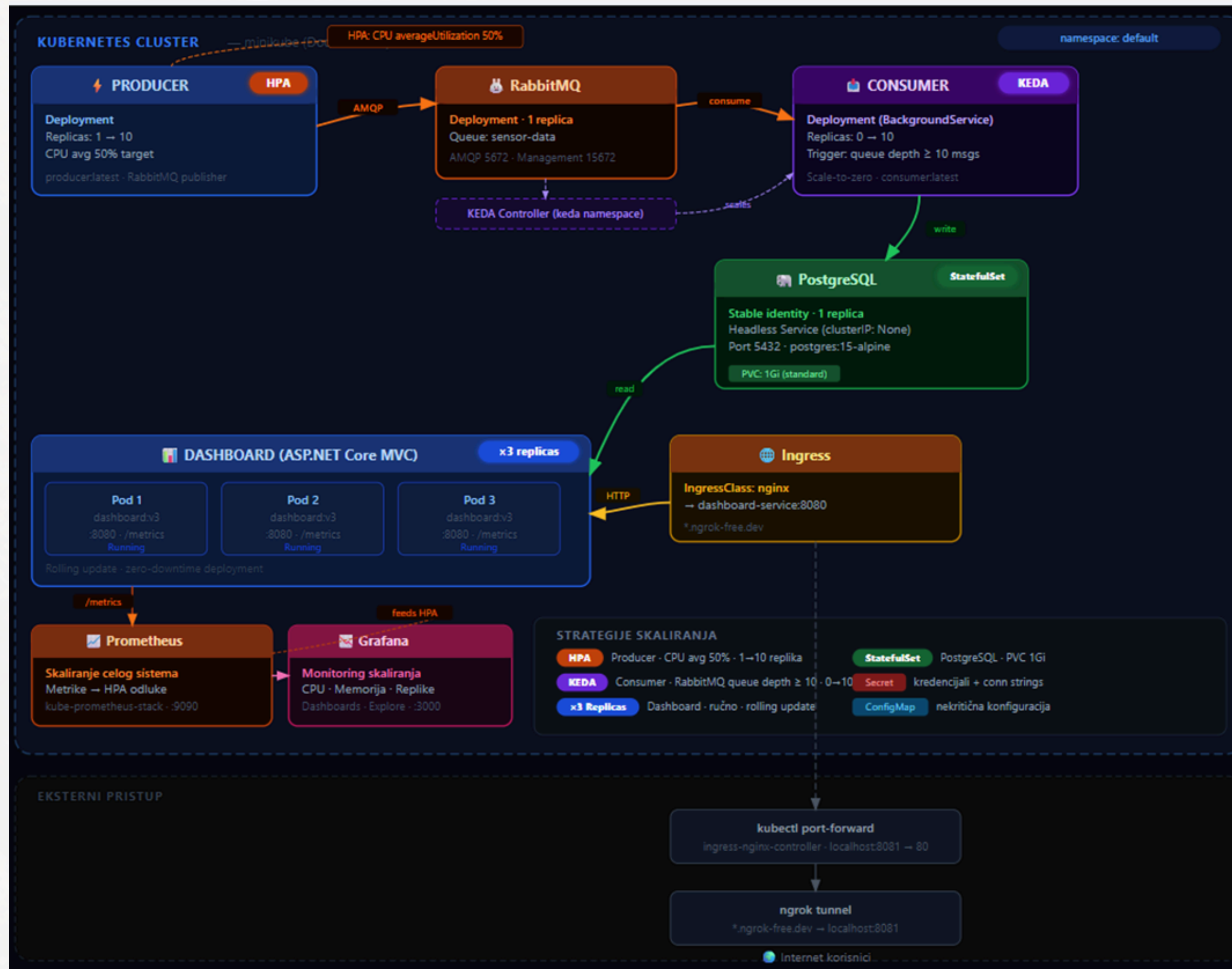
```
consumer > secret.yaml
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: consumer-secret
5  type: Opaque
6  data:
7    RabbitMQ__Username: dXNlcg==
8    RabbitMQ__Password: cGFzc3dvcmQ=
9    RabbitMQ__HostString: YW1xcDovL3VzZXI6cGFzc3dvcmRAcmFz
10   ConnectionStrings__Database: SG9zdD1wb3N0Z3JlcztQb3J0
```



Secret

- Kubernetes objekat namenjen za čuvanje osetljivih podataka.
- Podaci u Secret-u su Base64 kodirani.

Arhitektura sistema



Producer – Konfiguracioni i secret fajlovi

Servis koji čita senzorske podatke iz txt fajla i šalje ih u red poruka.

```
Infrastructure > k8s > producer > config-map.yaml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: producer-config-map
5  data:
6    seq_host: "http://seq-service:5341/"
7    file_path: "/app/data/iot_telemetry_data.csv"
```

```
Infrastructure > k8s > producer > secret.yaml
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: producer-secret
5  type: Opaque
6  data:
7    RabbitMQ__Username: dXNlcmg==
8    RabbitMQ__Password: cGFzc3dvcmQ=
9    RabbitMQ__Host: cmFiYml0bXEtc2Vydm1jZQ==
```

```
Infrastructure > k8s > producer > stream-config.yaml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: stream-config
5  data:
6    appsettings.json: |
7      {
8        "AppSettings" : {
9          "StreamDelay": 100
10       }
11     }
```

stream-config je Config map koji producer mountuje kao volume - unutar poda je dostupan je kao fajl /config/appsettings.json sa jednim podešavanjem: StreamDelay, koji kontroliše koliko ms producer čeka između slanja poruka na RabbitMQ. Ovo je fajl koji se može menjati u toku vremena i sa kubectl apply -f producer automatski hvata promenu i konfiguriše brzinu slanja

Producer – Deployment

```
Infrastructure > k8s > producer > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: producer-deployment
5    labels:
6      app: producer-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: producerapp
12   template:
13     metadata:
14       labels:
15         app: producerapp
16     spec:
17       containers:
18         - name: producerapp
19           image: producer:latest
20           imagePullPolicy: Never
21           ports:
22             - containerPort: 8080
23           env:
24             - name: DOTNET_USE_POLLING_FILE_WATCHER
25               value: "true"
26             - name: Seq_ServerUrl
27               valueFrom:
28                 configMapKeyRef:
29                   name: producer-config-map
30                   key: seq_host
31             - name: AppSettings__FilePath
32               valueFrom:
33                 configMapKeyRef:
34                   name: producer-config-map
35                   key: file_path
```

```
36     - name: RabbitMQ__Host
37       valueFrom:
38         secretKeyRef:
39           name: producer-secret
40           key: RabbitMQ__Host
41     - name: RabbitMQ__Username
42       valueFrom:
43         secretKeyRef:
44           name: producer-secret
45           key: RabbitMQ__Username
46     - name: RabbitMQ__Password
47       valueFrom:
48         secretKeyRef:
49           name: producer-secret
50           key: RabbitMQ__Password
51     volumeMounts:
52       - name: stream-config
53         mountPath: /config
54     resources:
55       requests:
56         cpu: "100m"
57         memory: "128Mi"
58       limits:
59         cpu: "500m"
60         memory: "256Mi"
61     volumes:
62       - name: stream-config
63         configMap:
64           name: stream-config
```

1. volumes se pravi na osnovu config-map i mountuje se u kontejner na putanji /config.
2. Definišu se resursi odnosno minimalni i maksimalni resursi koje Kubernetes garantuje podu pri schedulingu.

Producer – Horizontal pod Autoscaler

Infrastructure > k8s > producer > hpa.yaml

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: producer-hpa
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind: Deployment
9      name: producer-deployment
10   minReplicas: 1
11   maxReplicas: 10
12   metrics:
13     - type: Resource
14       resource:
15         name: cpu
16         target:
17           type: Utilization
18           averageUtilization: 50
```

HPA - prati prosečnu upotrebu CPU svih producer prodova i automatski menja broj replika između 1 i 10. Logika skaliranja - Kada prosečna CPU upotreba prekorači 50% od requests što je (50% od 100m = 50m po podu), HPA dodaje nove replike dok prosek ne padne ispod ovog praga. Kada opterećenje padne smanjuje replike nazad ka 1.

Consumer – konfiguracioni fajlovi

```
Infrastructure > k8s > consumer > config-map.yaml
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: consumer-config-map
5  data:
6    rabbitmq_host: "rabbitmq-service"
7    seq_host: "http://seq-service:5341/"
```

```
Infrastructure > k8s > consumer > secret.yaml
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: consumer-secret
5  type: Opaque
6  data:
7    RabbitMQ__Username: dXNlcg==
8    RabbitMQ__Password: cGFzc3dvcmQ=
9    RabbitMQ__HostString: YW1xcDovL3VzZXI6cGFzc3dvcmRAcmFiYml0b
10 | ConnectionStrings__Database: SG9zdD1wb3N0Z3JlcztQb3J0PTU0Mz
```

Consumer – Deployment

```
Infrastructure > k8s > consumer > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: consumer-deployment
5    labels:
6      app: consumer-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: consumerapp
12    template:
13     metadata:
14       labels:
15         app: consumerapp
16     spec:
17       containers:
18       - name: consumerapp
19         image: consumer:latest
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 8080
23         env:
24         - name: RabbitMQ_Host
25           valueFrom:
26             configMapKeyRef:
27               name: consumer-config-map
28               key: rabbitmq_host
29         - name: Seq_Url
30           valueFrom:
31             configMapKeyRef:
32               name: consumer-config-map
33               key: seq_host
34         - name: RabbitMQ_Username
35           valueFrom:
36             secretKeyRef:
37               name: consumer-secret
38               key: RabbitMQ_Username
39         - name: RabbitMQ_Password
40           valueFrom:
41             secretKeyRef:
42               name: consumer-secret
43               key: RabbitMQ_Password
44         - name: ConnectionStrings_Database
45           valueFrom:
46             secretKeyRef:
47               name: consumer-secret
48               key: ConnectionStrings_Database
```

Consumer – Deployment

```
Infrastructure > k8s > consumer > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: consumer-deployment
5    labels:
6      app: consumer-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: consumerapp
12    template:
13     metadata:
14       labels:
15         app: consumerapp
16     spec:
17       containers:
18       - name: consumerapp
19         image: consumer:latest
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 8080
23         env:
24         - name: RabbitMQ_Host
25           valueFrom:
26             configMapKeyRef:
27               name: consumer-config-map
28               key: rabbitmq_host
29         - name: Seq_Url
30           valueFrom:
31             configMapKeyRef:
32               name: consumer-config-map
33               key: seq_host
34         - name: RabbitMQ_Username
35           valueFrom:
36             secretKeyRef:
37               name: consumer-secret
38               key: RabbitMQ_Username
39         - name: RabbitMQ_Password
40           valueFrom:
41             secretKeyRef:
42               name: consumer-secret
43               key: RabbitMQ_Password
44         - name: ConnectionStrings__Database
45           valueFrom:
46             secretKeyRef:
47               name: consumer-secret
48               key: ConnectionStrings__Database
```

Consumer – KEDA horizontal scaling

Infrastructure > k8s > consumer > scaled-object.yaml

```
1  apiVersion: keda.sh/v1alpha1
2  kind: ScaledObject
3  metadata:
4    name: consumer-scale
5  spec:
6    scaleTargetRef:
7      name: consumer-deployment
8    minReplicaCount: 0
9    maxReplicaCount: 10
10   triggers:
11   - type: rabbitmq
12     metadata:
13       queueName: "sensor-data"
14       queueLength: "10"
15     authenticationRef:
16       name: rabbitmq-trigger-auth
```

KEDA - Kubernetes event driven autoscaling - omogućava skaliranje aplikacija na osnovu događaja. Čim stigne 10 poruka u redu dodaje se još jedna replika do maksimalno 10 replika. Kada nema poruka u redu broj replika je 0. Koristi objekat rabbitmq-trigger-auth koji sadrži kredencijale za pristup rabbitmq.

Infrastructure > k8s > consumer > trigger-auth.yaml

```
1  apiVersion: keda.sh/v1alpha1
2  kind: TriggerAuthentication
3  metadata:
4    name: rabbitmq-trigger-auth
5  spec:
6    secretTargetRef:
7      - parameter: host
8        name: consumer-secret
9        key: RabbitMQ_HostString
```

Dashboard - Deployment

```
Infrastructure > k8s > dashboard > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: dashboard-deployment
5    labels:
6      app: dashboard-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: dashboardapp
12    template:
13     metadata:
14       labels:
15         app: dashboardapp
16     spec:
17       containers:
18       - name: dashboardapp
19         image: dashboard:latest
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 8080
23         env:
24         - name: ConnectionStrings__Database
25           valueFrom:
26             secretKeyRef:
27               name: dashboard-secret
28               key: ConnectionStrings__Database
29         resources:
30           requests:
31             cpu: "100m"
32             memory: "256Mi"
33           limits:
34             cpu: "500m"
35             memory: "1Gi"
```

```
36  ---
37  apiVersion: v1
38  kind: Service
39  metadata:
40    name: dashboard-service
41    labels:
42      app: dashboardapp
43  spec:
44    type: NodePort
45    selector:
46      app: dashboardapp
47    ports:
48    - protocol: TCP
49      port: 8080
50      targetPort: 8080
51      name: http
```

Dashboard - Deployment

```
Infrastructure > k8s > dashboard > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: dashboard-deployment
5    labels:
6      app: dashboard-deployment
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: dashboardapp
12    template:
13     metadata:
14       labels:
15         app: dashboardapp
16     spec:
17       containers:
18       - name: dashboardapp
19         image: dashboard:latest
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 8080
23         env:
24         - name: ConnectionStrings__Database
25           valueFrom:
26             secretKeyRef:
27               name: dashboard-secret
28               key: ConnectionStrings__Database
29         resources:
30           requests:
31             cpu: "100m"
32             memory: "256Mi"
33           limits:
34             cpu: "500m"
35             memory: "1Gi"
```

```
36  ---
37  apiVersion: v1
38  kind: Service
39  metadata:
40    name: dashboard-service
41    labels:
42      app: dashboardapp
43  spec:
44    type: NodePort
45    selector:
46      app: dashboardapp
47    ports:
48    - protocol: TCP
49      port: 8080
50      targetPort: 8080
51      name: http
```

Dashboard – Ingress

Omogućava spoljašnji pristup aplikaciji preko HTTP protokola.

```
Infrastructure > k8s > dashboard > ingress.yaml
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: dashboard-ingress
5    namespace: default
6  spec:
7    ingressClassName: nginx
8    rules:
9      - host: riding-handbrake-upgrade.ngrok-free.dev
10      http:
11        paths:
12          - path: /
13            pathType: Prefix
14            backend:
15              service:
16                name: dashboard-service
17                port:
18                  number: 8080
```

Dashboard – VerticalPodAutoscaler

```
dashboard > scaled-object.yaml
1  apiVersion: autoscaling.k8s.io/v1
2  kind: VerticalPodAutoscaler
3  metadata:
4    name: dashboard-vpa
5  spec:
6    targetRef:
7      apiVersion: "apps/v1"
8      kind: Deployment
9      name: dashboard-deployment
10   updatePolicy:
11     updateMode: "InPlaceOrRecreate"
12   resourcePolicy:
13     containerPolicies:
14     - containerName: "dashboardapp"
15       minAllowed:
16         cpu: 100m
17         memory: 128Mi
18       maxAllowed:
19         cpu: 2
20         memory: 2Gi
21     controlledResources:
22     - cpu
23     - memory
```

Dashboard - Secret

Infrastructure > k8s > dashboard >  secret.yaml

```
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    | name: dashboard-secret
5  type: Opaque
6  data:
7    | ConnectionStrings__Database: SG9zdD1wb3N0Z3
8
```

Seq

```
Infrastructure > k8s > seq > deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: seq-deployment
5    labels:
6      app: seq
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: seq
12    template:
13     metadata:
14       labels:
15         app: seq
16     spec:
17       containers:
18         - name: seq
19           image: datalust/seq:latest
20           env:
21             - name: ACCEPT_EULA
22               value: "Y"
23           ports:
24             - containerPort: 80
25             - containerPort: 5341
26 ---
27 apiVersion: v1
28 kind: Service
29 metadata:
30   name: seq-service
31 spec:
32   selector:
33     app: seq
34   ports:
35     - name: ui
36       port: 80
37       targetPort: 80
38     - name: ingestion
39       port: 5341
40       targetPort: 5341
```

RabbitMQ

```
Infrastructure > k8s > rabbitmq > rabbitmq-deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: rabbitmq-deployment
5  spec:
6    replicas: 1
7    selector:
8      matchLabels:
9        app: rabbitmq
10   template:
11     metadata:
12       labels:
13         app: rabbitmq
14     spec:
15       containers:
16         - name: rabbitmq
17           image: rabbitmq:3-management
18           ports:
19             - containerPort: 5672
20             - containerPort: 15672
21           env:
22             - name: RABBITMQ_DEFAULT_USER
23               value: user
24             - name: RABBITMQ_DEFAULT_PASS
25               value: password
26
27   ---
28   apiVersion: v1
29   kind: Service
30   metadata:
31     name: rabbitmq-service
32   spec:
33     selector:
34       app: rabbitmq
35     ports:
36       - name: amqp
37         port: 5672
38       - name: management
39         port: 15672
40   type: ClusterIP
```

Postgres

```
Infrastructure > k8s > postgres > secret.yaml
```

```
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: postgres-secret
5  type: Opaque
6  data:
7    POSTGRES_PASSWORD: cG9zdGdyZXM=
8    POSTGRES_USER: cG9zdGdyZXM=
9    POSTGRES_DB: bXlkYg==
```

```
Infrastructure > k8s > postgres > statefulset.yaml
```

```
1  apiVersion: apps/v1
2  kind: StatefulSet
3  metadata:
4    name: postgres
5  spec:
6    serviceName: "postgres"
7    replicas: 1
8    selector:
9      matchLabels:
10       app: postgresapp
11    template:
12      metadata:
13        labels:
14          app: postgresapp
15      spec:
16        containers:
17          - name: postgres
18            image: postgres:15-alpine
19            ports:
20              - containerPort: 5432
21                name: postgres
22            volumeMounts:
23              - name: postgres
24                mountPath: /var/lib/postgresql/data
25            env:
26              - name: POSTGRES_PASSWORD
27                valueFrom:
28                  secretKeyRef:
29                    name: postgres-secret
30                    key: POSTGRES_PASSWORD
31              - name: POSTGRES_USER
32                valueFrom:
33                  secretKeyRef:
34                    name: postgres-secret
35                    key: POSTGRES_USER
36              - name: POSTGRES_DB
37                valueFrom:
38                  secretKeyRef:
39                    name: postgres-secret
40                    key: POSTGRES_DB
```

```
41  volumeClaimTemplates:
42    - metadata:
43        name: postgres
44      spec:
45        accessModes: [ "ReadWriteOnce" ]
46        storageClassName: "standard"
47        resources:
48          requests:
49            storage: 1Gi
50  ---
51
52  apiVersion: v1
53  kind: Service
54  metadata:
55    name: postgres
56    labels:
57      app: postgresapp
58  spec:
59    clusterIP: None
60    selector:
61      app: postgresapp
62    ports:
63      - protocol: TCP
64        port: 5433
65        targetPort: 5432
66      name: postgres
```

Demo

```
PS C:\Users\Veljko\Desktop\SIR> kubectl get hpa producer-hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
producer-hpa	Deployment/producer-deployment	cpu: <unknown>/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: <unknown>/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: 7%/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: 3%/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: 37%/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: 137%/50%	1	10	1	13d
producer-hpa	Deployment/producer-deployment	cpu: 137%/50%	1	10	3	13d
producer-hpa	Deployment/producer-deployment	cpu: 61%/50%	1	10	3	13d
producer-hpa	Deployment/producer-deployment	cpu: 62%/50%	1	10	3	13d
producer-hpa	Deployment/producer-deployment	cpu: 62%/50%	1	10	4	13d
producer-hpa	Deployment/producer-deployment	cpu: 65%/50%	1	10	4	14d
producer-hpa	Deployment/producer-deployment	cpu: 57%/50%	1	10	4	14d
producer-hpa	Deployment/producer-deployment	cpu: 57%/50%	1	10	5	14d
producer-hpa	Deployment/producer-deployment	cpu: 49%/50%	1	10	5	14d
producer-hpa	Deployment/producer-deployment	cpu: 69%/50%	1	10	5	14d
producer-hpa	Deployment/producer-deployment	cpu: 69%/50%	1	10	7	14d
producer-hpa	Deployment/producer-deployment	cpu: 32%/50%	1	10	7	14d
producer-hpa	Deployment/producer-deployment	cpu: 31%/50%	1	10	7	14d
producer-hpa	Deployment/producer-deployment	cpu: 36%/50%	1	10	7	14d

```
PS C:\Users\Veljko\Desktop\SIR\infrastructure\k8s\producer> kubectl get pods -l app=producerapp
```

NAME	READY	STATUS	RESTARTS	AGE
producer-deployment-5c8bbd9cc-56k4n	1/1	Running	0	2m25s
producer-deployment-5c8bbd9cc-kj7wm	1/1	Running	0	4m25s
producer-deployment-5c8bbd9cc-kxcrq	1/1	Running	0	6m26s
producer-deployment-5c8bbd9cc-s9rhl	1/1	Running	0	13m
producer-deployment-5c8bbd9cc-tn7sz	1/1	Running	0	8m26s
producer-deployment-5c8bbd9cc-wjlsz	1/1	Running	0	8m26s
producer-deployment-5c8bbd9cc-xmcpz	1/1	Running	0	2m25s

— Skaliranje Producer podova

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get pods -l app=producerapp -w
```

NAME	READY	STATUS	RESTARTS	AGE
producer-deployment-fbdf7449b-bzfnb	1/1	Running	0	8m28s

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get hpa producer-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
producer-hpa	Deployment/producer-deployment	cpu: 1%/50%	1	10	1	4d21h

Kada je StreamDelay na 10s dovoljan je jedan pod,
Kada se smanji na 1s onda se broj podova
povećava..

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get hpa producer-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
producer-hpa	Deployment/producer-deployment	cpu: 80%/50%	1	10	1	4d22h

Nakon povećanja vidi se da se cpu usage producera
povećao i sada je na 80% i moraće hpa da odradi
skaliranje

— Skaliranje Producer podova

NAKON SKALIRANJA:

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get hpa producer-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
producer-hpa	Deployment/producer-deployment	cpu: 43%/50%	1	10	4	4d22h

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get pods -l app=producerapp -w
```

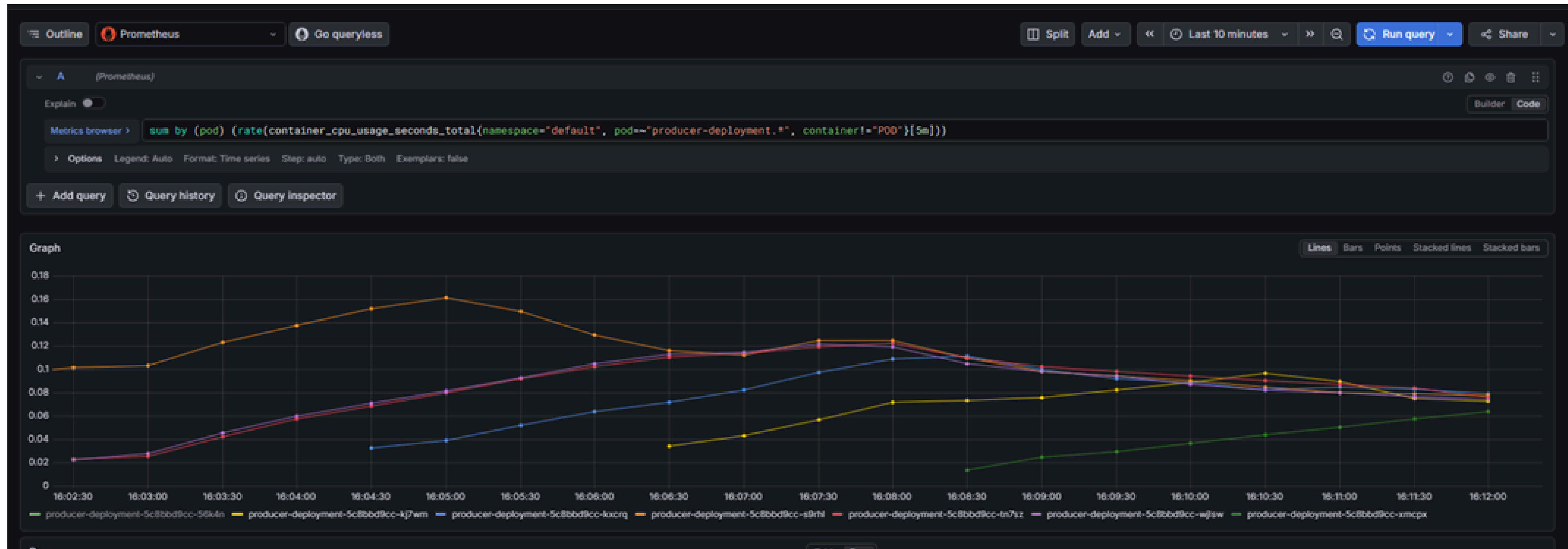
NAME	READY	STATUS	RESTARTS	AGE
producer-deployment-fbdf7449b-bzfnb	1/1	Running	0	25m
producer-deployment-fbdf7449b-hh5d2	1/1	Running	0	3m8s
producer-deployment-fbdf7449b-kbrk7	1/1	Running	0	7m9s
producer-deployment-fbdf7449b-zmdl8	1/1	Running	0	3m8s

— Skaliranje Producer podova



U grafani se vidi da su bila 2 poda (zeleni i plavi) i trošili su 0.10-0.12 cores a prag je 50% od 0.100 dakle 0.050. Zbog toga se dodaju još dva poda žuta i zelena i CPU im raste dok ne stigne niv o starih replika. To je HPA skaliranje na delu.

Skaliranje Producer podova



Kasnije se broj podova još više povećao...

– Skaliranje Consumera

Inicijalno kada se poruke šalju na 10 sekunde u queue dovoljan je jedan consumer.

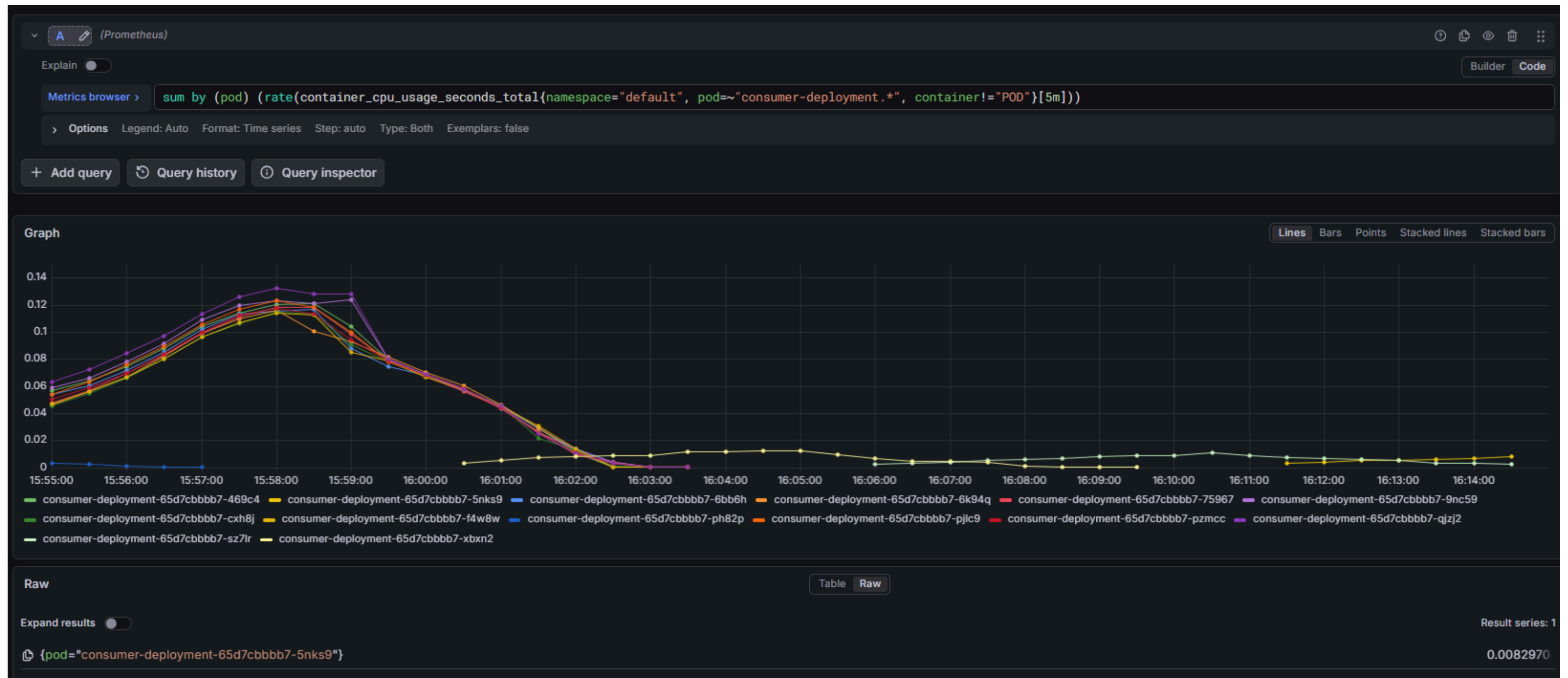
```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get pods -l app=consumerapp -w
NAME                                READY   STATUS    RESTARTS   AGE
consumer-deployment-68dcf5c9c6-nfp44 1/1     Running   0          4m38s
```

Kada se brzina slanja poveća, povećava se i broj poruka u redu i dolazi do skaliranja consumera.

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	sensor-data	classic	D	running	95	32	127	141/s	116/s	110/s	

```
PS C:\Users\Veljko\Desktop\SIR\Infrastructure\k8s> kubectl get pods -l app=consumerapp -w
NAME                                READY   STATUS    RESTARTS   AGE
consumer-deployment-65d7cbbbb7-2rfjz 1/1     Running   0          25s
consumer-deployment-65d7cbbbb7-52bdr 1/1     Running   0          25s
consumer-deployment-65d7cbbbb7-5fs52 1/1     Running   0          10s
consumer-deployment-65d7cbbbb7-5vkz1 1/1     Running   0          10s
consumer-deployment-65d7cbbbb7-g28ft 1/1     Running   0          10s
consumer-deployment-65d7cbbbb7-hf8nr 1/1     Running   0          25s
consumer-deployment-65d7cbbbb7-hz74q 1/1     Running   0          10s
consumer-deployment-65d7cbbbb7-tkc4r 1/1     Running   0          5m11s
```

– Skaliranje Consumera



— Dashboard Service - k6 stress test

Infrastructure > tests > stress > `dashboard-service.js` > ...

```
1  import http from 'k6/http';
2  import { check } from 'k6';
3
4  export const options = {
5    stages: [
6      { duration: '30s', target: 10 },
7      { duration: '2m', target: 10 },
8
9      { duration: '30s', target: 20 },
10     { duration: '2m', target: 20 },
11
12     { duration: '1m', target: 0 }
13   ],
14   thresholds: {
15     http_req_failed: ['rate<0.01'],
16     http_req_duration: ['p(95)<15000'],
17   },
18 };
19
20 export default function () {
21   const res = http.get('http://127.0.0.1:52031/api/report/export');
22
23   check(res, {
24     'status is 200': (r) => r.status === 200,
25   });
26 }
```

k6 stress test koji simulira rastuće opterećenje na `/api/cpu-stress` endpointu dashboard servisa.

Faze (~6 minuta ukupno):

- Ramp-up na 10 korisnika za 30s, drži 2 minuta
- Ramp-up na 20 korisnika za 30s, drži 2 minuta
- Ramp-down na 0 za 1 minutu

Thresholds (kriterijumi prolaza):

- Manje od 1% failovanih requestova
- 95% requestova završi ispod 15 sekundi

Šta svaki korisnik radi: neprekidno šalje GET requestove na `/api/report/export`

— Dashboard Service - k6 stress test

```
THRESHOLDS

http_req_duration
✓ 'p(95)<15000' p(95)=4.52s

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 2304    6.399435/s
checks_succeeded...: 100.00% 2304 out of 2304
checks_failed.....: 0.00%   0 out of 2304

✓ status is 200

HTTP
http_req_duration.....: avg=2.09s min=112.46ms med=1.79s max=6.9s p(90)=3.9
95)=4.52s
{ expected_response:true }....: avg=2.09s min=112.46ms med=1.79s max=6.9s p(90)=3.9
95)=4.52s
http_req_failed.....: 0.00%   0 out of 2304
http_reqs.....: 2304    6.399435/s

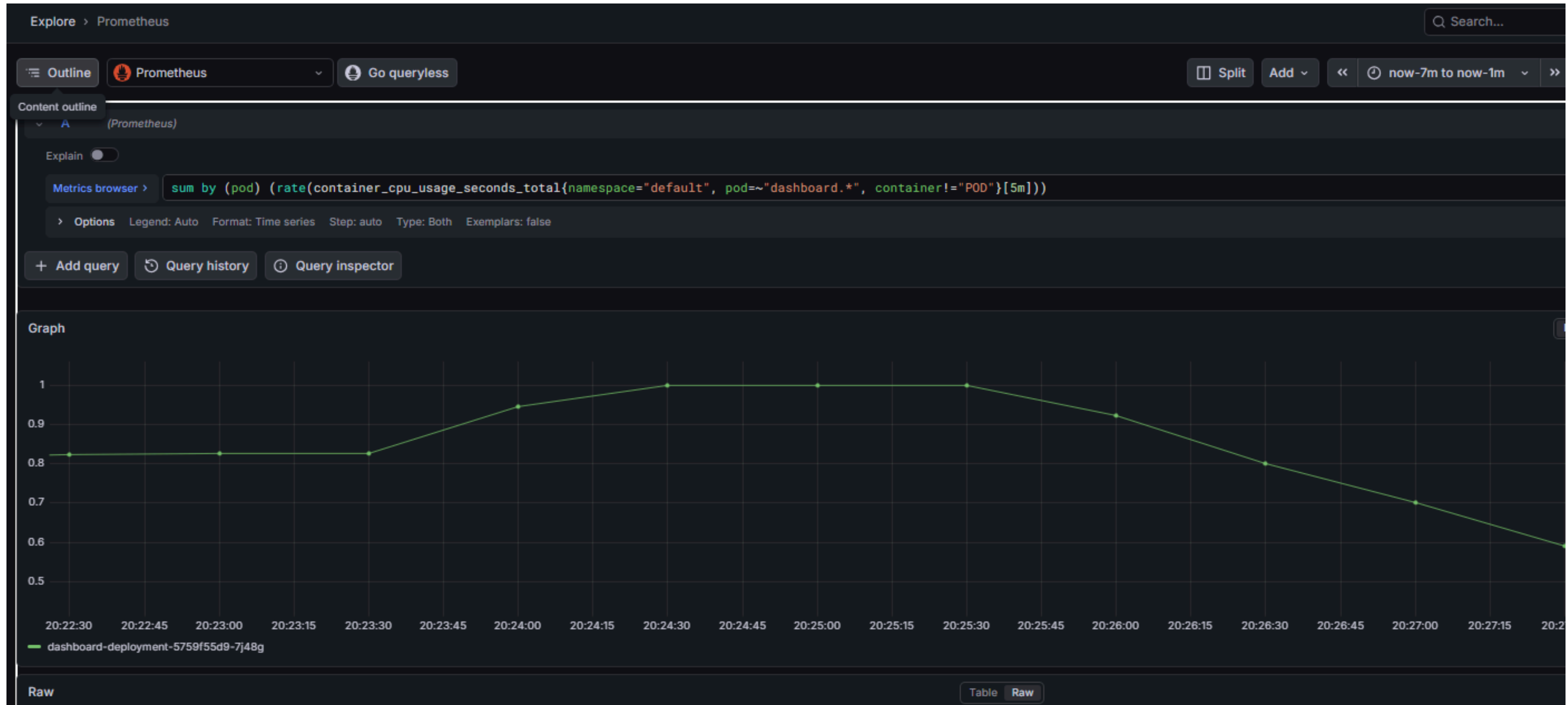
EXECUTION
iteration_duration.....: avg=2.09s min=113.5ms  med=1.79s max=6.9s p(90)=3.9
95)=4.52s
iterations.....: 2304    6.399435/s
vus.....: 1      min=1      max=20
vus_max.....: 20    min=20    max=20

NETWORK
data_received.....: 741 MB 2.1 MB/s
data_sent.....: 203 kB 563 B/s
```

```
C:\Users\Veljko\Desktop\SIR>kubectl get vpa dashboard-vpa -w
NAME                MODE                CPU    MEM    PROVIDED    AGE
dashboard-vpa       InPlaceOrRecreate   247m   250Mi   True        99s
dashboard-vpa       InPlaceOrRecreate   511m   351198544   True        2m24s
dashboard-vpa       InPlaceOrRecreate   587m   410771395   True        4m24s
dashboard-vpa       InPlaceOrRecreate   587m   587804717   True        5m24s
```

vidi se da je vpa skalirao cpu na 587m sa 100m
i skalirao je memoriju sa 128Mi na 2Gi

— Dashboard Service - k6 stress test



— Producer - Self healing

```
PS C:\Users\Veljko\Desktop\SIR> kubectl get pods -l app=producerapp
NAME                                READY   STATUS    RESTARTS   AGE
producer-deployment-fbdf7449b-bzfnb 1/1     Running   3 (3m49s ago)  24h
producer-deployment-fbdf7449b-tggvw 1/1     Running   0           6s
producer-deployment-fbdf7449b-xcv4t 1/1     Running   0           6s
PS C:\Users\Veljko\Desktop\SIR> kubectl delete pod producer-deployment-fbdf7449b-bzfnb
pod "producer-deployment-fbdf7449b-bzfnb" deleted from default namespace
PS C:\Users\Veljko\Desktop\SIR> kubectl get pods -l app=producerapp
NAME                                READY   STATUS    RESTARTS   AGE
producer-deployment-fbdf7449b-gbps7 1/1     Running   0           5s
producer-deployment-fbdf7449b-tggvw 1/1     Running   0           2m1s
producer-deployment-fbdf7449b-xcv4t 1/1     Running   0           2m1s
```

— Dashboard - Ingress

Za implementaciju ingressa koristi se ngrok uz pomoć njega sam lokalnu aplikaciju izložio na internet. Neophodno je bilo da se ingress controller izloži na lokalni port.

```
Command Prompt - ngrok ht X + v
ngrok
► ngrok is way more than tunnels to localhost. https://ngrokhq.link/video

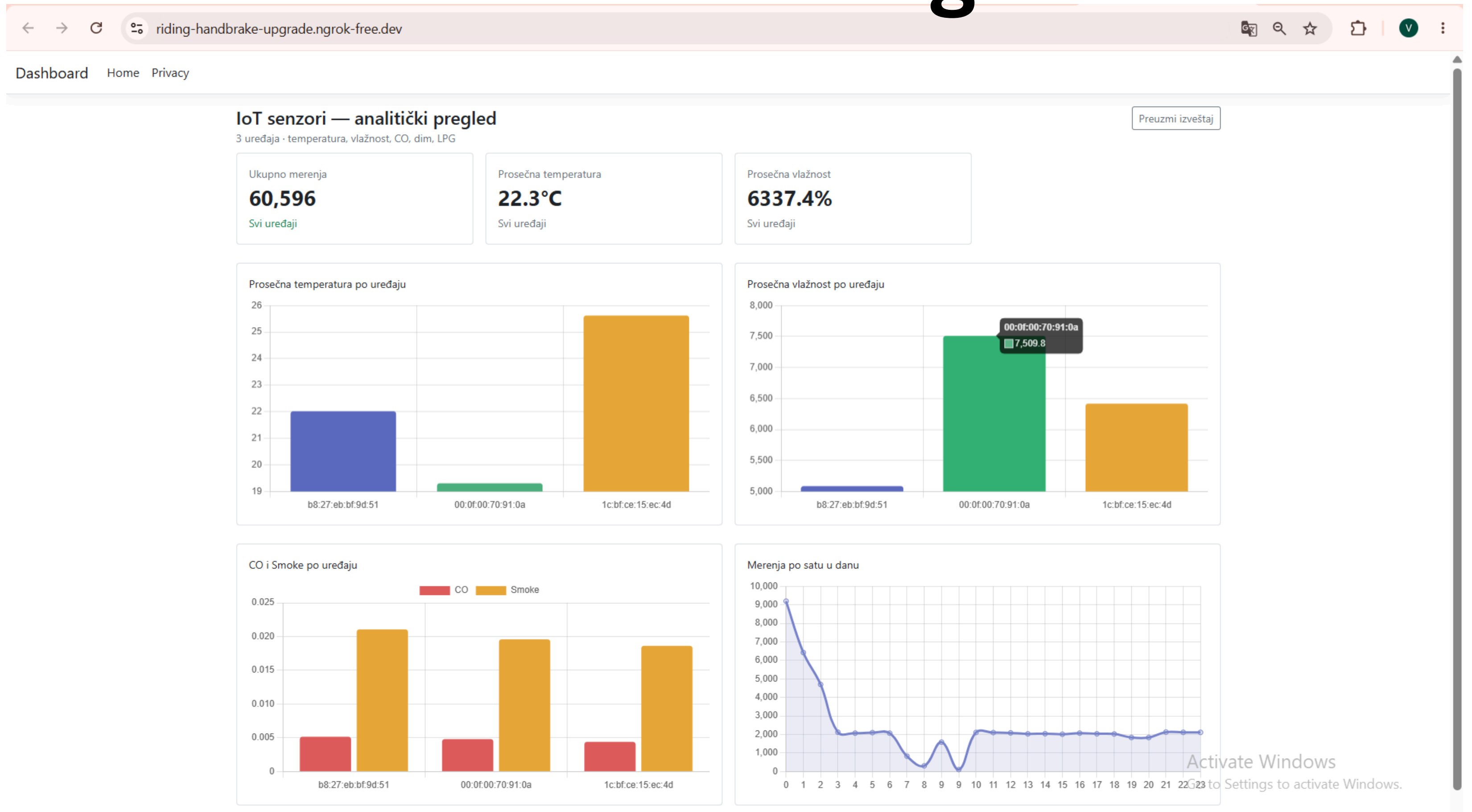
Session Status      online
Account             veljkoveljovic13@gmail.com (Plan: Free)
Update              update available (version 3.39.9, Ctrl-U to update)
Version             3.39.8
Region              Europe (eu)
Latency             87ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://riding-handbrake-upgrade.ngrok-free.dev -> http://localhost:8081

Connections          ttl      opn      rt1      rt5      p50      p90
                    6        0        0.00     0.01     1.02     76.81

HTTP Requests
-----
14:19:06.098 CEST GET /favicon.ico 200 OK
14:19:04.794 CEST GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js 200 OK
14:19:04.263 CEST GET / 200 OK
14:19:04.797 CEST GET /Dashboard.styles.css 200 OK
14:19:04.793 CEST GET /lib/bootstrap/dist/css/bootstrap.min.css 200 OK
14:19:04.794 CEST GET /lib/jquery/dist/jquery.min.js 200 OK
14:19:04.794 CEST GET /js/site.js 200 OK
14:19:04.794 CEST GET /css/site.css 200 OK
```

```
PS C:\Users\Veljko\Desktop\SIR> kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
Handling connection for 8081
Handling connection for 8081
```

Dashboard - Ingress



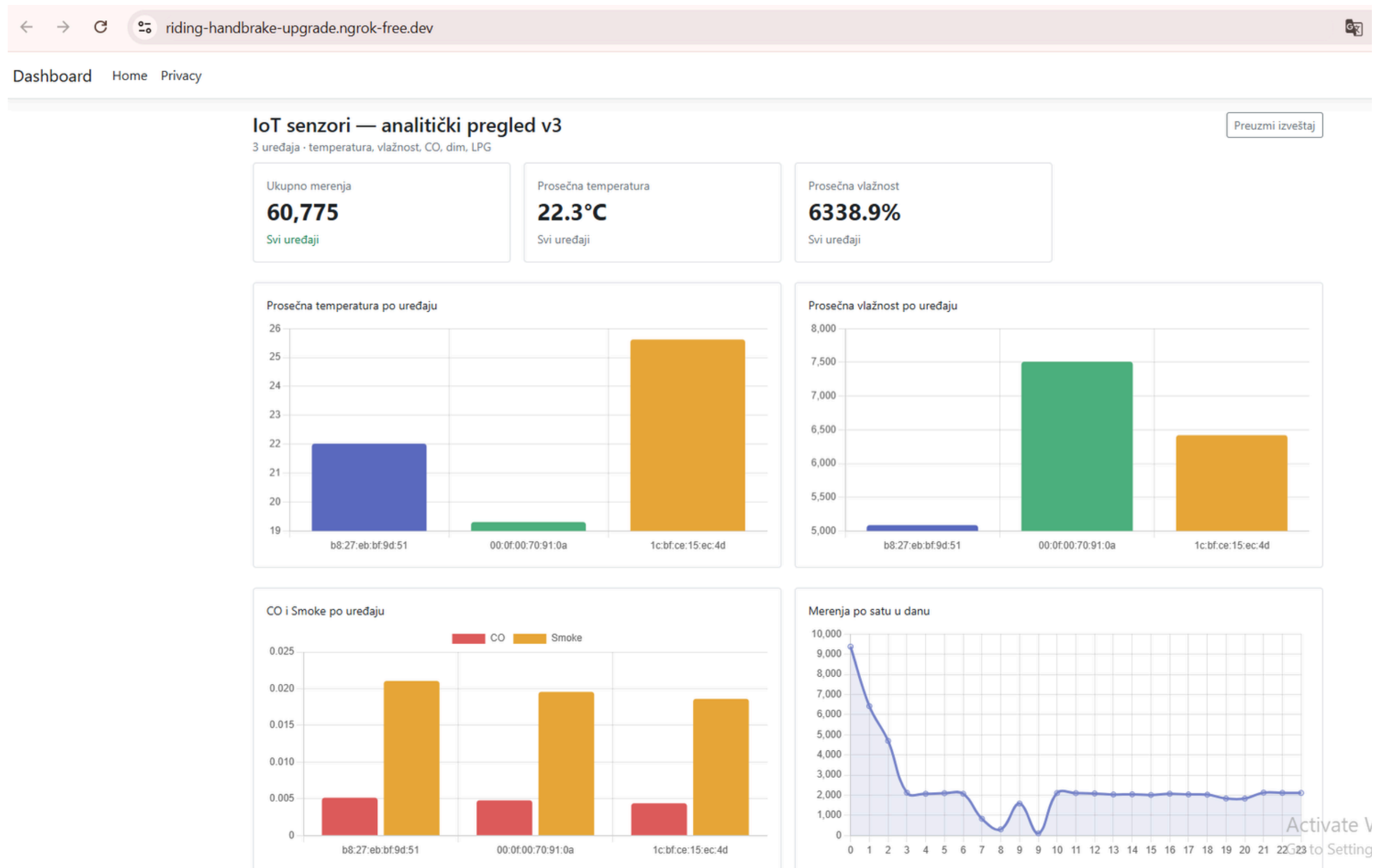
Dashboard - Rolling Update

```
PS C:\Users\Veljko\Desktop\SIR> kubectl get pods -l app=dashboardapp -w
NAME                                READY   STATUS    RESTARTS   AGE
dashboard-deployment-6b4487d697-j7vxn 1/1     Running   0          7m25s
dashboard-deployment-6b4487d697-r5k5l 1/1     Running   0          3s
dashboard-deployment-6b4487d697-v6bnq 1/1     Running   0          3s
dashboard-deployment-68786bf8d9-q7rrv 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-q7rrv 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-q7rrv 0/1     ContainerCreating 0          0s
dashboard-deployment-68786bf8d9-q7rrv 1/1     Running   0          2s
dashboard-deployment-6b4487d697-r5k5l 1/1     Terminating 0          40s
dashboard-deployment-6b4487d697-r5k5l 1/1     Terminating 0          40s
dashboard-deployment-68786bf8d9-nbt4c 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-nbt4c 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-nbt4c 0/1     ContainerCreating 0          0s
dashboard-deployment-6b4487d697-r5k5l 0/1     Completed 0          40s
dashboard-deployment-6b4487d697-r5k5l 0/1     Completed 0          41s
dashboard-deployment-6b4487d697-r5k5l 0/1     Completed 0          41s
dashboard-deployment-68786bf8d9-nbt4c 1/1     Running   0          1s
dashboard-deployment-6b4487d697-v6bnq 1/1     Terminating 0          41s
dashboard-deployment-6b4487d697-v6bnq 1/1     Terminating 0          41s
dashboard-deployment-68786bf8d9-fkzwh 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-fkzwh 0/1     Pending   0          0s
dashboard-deployment-68786bf8d9-fkzwh 0/1     ContainerCreating 0          0s
dashboard-deployment-6b4487d697-v6bnq 0/1     Completed 0          41s
dashboard-deployment-6b4487d697-v6bnq 0/1     Completed 0          42s
dashboard-deployment-6b4487d697-v6bnq 0/1     Completed 0          42s
dashboard-deployment-68786bf8d9-fkzwh 1/1     Running   0          2s
dashboard-deployment-6b4487d697-j7vxn 1/1     Terminating 0          8m5s
dashboard-deployment-6b4487d697-j7vxn 1/1     Terminating 0          8m5s
dashboard-deployment-6b4487d697-j7vxn 0/1     Completed 0          8m5s
dashboard-deployment-6b4487d697-j7vxn 0/1     Completed 0          8m6s
dashboard-deployment-6b4487d697-j7vxn 0/1     Completed 0          8m6s
```

```
PS C:\Users\Veljko\Desktop\SIR> docker build -t dashboard:v3 -f ./Dashboard/Dockerfile ./Dashboard
[+] Building 14.3s (17/17) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default
=> => transferring dockerfile: 557B                                              0.0s
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0                0.7s
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0             0.7s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:63ebabdcde24cc8134304a6f50719cf1e22bb4e9f7148ac4ef96 0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 31.51kB                                             0.0s
=> [base 1/2] FROM mcr.microsoft.com/dotnet/aspnet:8.0@sha256:e78fda31142e28746a6908e288e0d40346793f691ca99d8d15 0.0s
=> CACHED [build 2/6] WORKDIR /src                                              0.0s
=> CACHED [build 3/6] COPY [Dashboard/Dashboard.csproj, Dashboard/]             0.0s
=> CACHED [build 4/6] RUN dotnet restore "Dashboard/Dashboard.csproj"           0.0s
=> [build 5/6] COPY . .                                                         0.2s
=> [build 6/6] RUN dotnet build "Dashboard/Dashboard.csproj" -c Release -o /app/build 11.3s
=> [publish 1/1] RUN dotnet publish "Dashboard/Dashboard.csproj" -c Release -o /app/publish 1.6s
=> CACHED [base 2/2] WORKDIR /app                                              0.0s
=> CACHED [final 1/2] WORKDIR /app                                             0.0s
=> [final 2/2] COPY --from=publish /app/publish .                             0.1s
=> exporting to image                                                           0.1s
=> => exporting layers                                                           0.1s
=> => writing image sha256:4515339b742bb1dbd57c7f915fe849a1551f1edaf48018767ad34600f57e5417 0.0s
=> => naming to docker.io/library/dashboard:v3                                0.0s
PS C:\Users\Veljko\Desktop\SIR> kubectl set image deployment/dashboard-deployment dashboardapp=dashboard:v3
deployment.apps/dashboard-deployment image updated
```

Postoje stari podovi. Nakon toga novi podovi prolaze kroz faze kreiranja. Kada su u fazi running stari podovi se gasu.

Dashboard - Rolling Update



— Prediktivno skaliranje pomoću mašinskog učenja

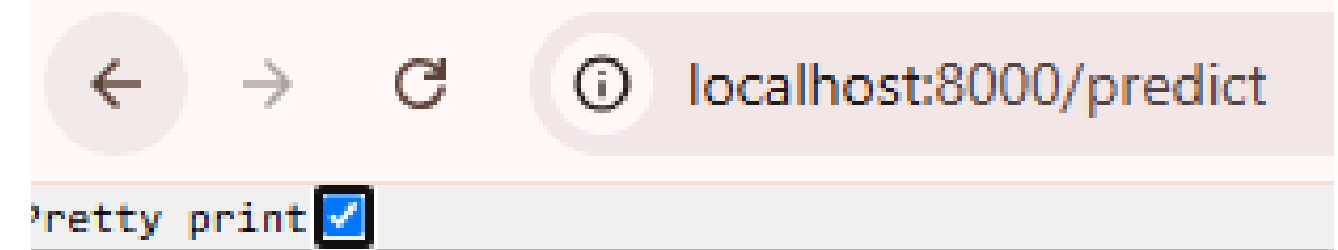
```
MLPredictor > train_v2.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.linear_model import LinearRegression
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.metrics import mean_absolute_error, r2_score
6 import pickle
7
8 df = pd.read_csv("queue_data_v2.csv")
9 print(f"Dataset: {len(df)} redova")
10 print(df.to_string(index=False))
11
12 X = df[["production_rate"]].values
13 y = df["min_consumers"].values
14
15 scaler = StandardScaler()
16 X_scaled = scaler.fit_transform(X)
17
18 model = LinearRegression()
19 model.fit(X_scaled, y)
20
21 y_pred = model.predict(X_scaled)
22 print(f"\nR2 score: {r2_score(y, y_pred):.4f}")
23 print(f"MAE: {mean_absolute_error(y, y_pred):.4f}")
24
25 for rate in [50, 100, 150, 200, 300, 400, 500]:
26     x_in = scaler.transform([[rate]])
27     pred = max(1, min(10, int(round(float(model.predict(x_in)[0])))))
28     print(f" rate={rate:5.0f} msg/s -> {pred} consumer(a)")
29
30 with open("model_v2.pkl", "wb") as f:
31     pickle.dump({"model": model, "scaler": scaler}, f)
32
33 print("\nModel saved: model_v2.pkl")
```

```
1 production_rate,queue_depth,min_consumers
2 225.6,30912,2
3 183.95,38586.0,2
4 175.5,43784,3
5 137.5,50852,2
6 290.25,58584.0,3
7 456.85,72960.0,10
```

```
60 with open("queue_data_v2.csv", "w", newline="") as f:
61     writer = csv.writer(f)
62     writer.writerow(["production_rate", "queue_depth", "min_consumers"])
63
64     for prod_replicas, delay in scenarios:
65         print(f"\n>>> Scenario: {prod_replicas} producera, delay={delay}ms")
66
67         set_producers(prod_replicas, delay)
68         set_consumers(0)
69         production_rate, base_depth = measure_rate(20)
70         print(f" Production rate: {production_rate:.1f} msg/s, depth: {base_depth:.0f}")
71
72         min_consumers = 10
73         found = False
74
75         for n in consumer_options:
76             print(f" Testiranje {n} consumer(a)...", end=" ", flush=True)
77             set_consumers(n)
78             rate, depth = measure_rate(30)
79             print(f"rate={rate:.1f} msg/s", flush=True)
80
81             if rate <= 5:
82                 min_consumers = n
83                 found = True
84                 print(f" -> Stabilan pri {n} consumer(a)")
85                 break
86
87         if not found:
88             print(f" -> Nije stabilizovano ni sa {consumer_options[-1]} consumer(a)")
89
90     writer.writerow([round(production_rate, 2), round(base_depth, 0), min_consumers])
91     f.flush()
```

— Prediktivno skaliranje pomoću mašinskog učenja

```
55 @app.get("/predict")
56 def predict():
57     current_depth = get_queue_depth_value()
58     now = time.time()
59     _history.append((now, current_depth))
60
61     if len(_history) < 2:
62         net_rate = 0.0
63     else:
64         oldest_time, oldest_depth = _history[0]
65         net_rate = (current_depth - oldest_depth) / (now - oldest_time)
66
67     current_consumers = get_current_consumers()
68     production_rate = max(0.0, net_rate + current_consumers * CONSUMER_THROUGHPUT_PER_REPLICA)
69
70     x = scaler.transform([[production_rate]])
71     raw = float(model.predict(x)[0])
72     replicas = max(1, min(10, int(round(raw))))
73
74     return {
75         "recommended_replicas": replicas,
76         "queue_depth": current_depth,
77         "net_rate_per_sec": round(net_rate, 2),
78         "estimated_production_rate": round(production_rate, 2),
79         "current_consumers": current_consumers,
80         "window_readings": len(_history)
81     }
```



```
[
  "recommended_replicas": 6,
  "queue_depth": 19466,
  "net_rate_per_sec": 336.11,
  "estimated_production_rate": 336.11,
  "current_consumers": 0,
  "window_readings": 2
]
```

```
84 @app.post("/scale")
85 def scale():
86     pred = predict()
87     replicas = pred["recommended_replicas"]
88
89     result = subprocess.run(
90         ["kubectl", "scale", "deployment", "consumer-deployment", f"--replicas={replicas}"],
91         capture_output=True, text=True
92     )
93
94     return {
95         "scaled_to": replicas,
96         "queue_depth": pred["queue_depth"],
97         "estimated_production_rate": pred["estimated_production_rate"],
98         "kubectl_output": result.stdout.strip() or result.stderr.strip()
99     }
```

The End

THANK YOU FOR LISTENING

Taylor Alonso

STUDIO SHODWE